

Delicias de Mamá

API REST y revisión con IA

Jafet Lavados y Matias Otero
Analista Programador · INACAP Punta Arenas
Docente: Paulo Simunovic Oyarzun

1. Problema y alcance

La PYME necesita ordenar su catálogo, solicitudes, disponibilidad y registros financieros. La solución ofrece un backend Django REST Framework y un portal sencillo para que el cliente se registre, solicite un producto y acepte una cotización. La dueña administra catálogo y finanzas mediante la API navegable y la administración de Django.

Esta entrega es independiente de Android y Supabase. Usa SQLite en desarrollo y contempla PostgreSQL para alojamiento. El objetivo evaluado es la API: autenticación, permisos, JSON, validación y operaciones CRUD.

2. Dominio y relaciones

Modelo	Responsabilidad y relación
Product	Catálogo, precio CLP y disponibilidad. Un producto tiene varios pedidos.
Day	Fecha, capacidad y bloqueo. Una fecha agrupa varios pedidos.
Order	Solicitud de un User para un Product y un Day (tres FK). Guarda estado, monto y versión de cotización.
Sale / Expense	Sale puede corresponder a un único Order (OneToOne). Expense puede asociarse a un Order (FK).
Audit	Registra actor User, acción, recurso y detalle; lectura exclusiva de la dueña.

Núcleo: User 1:N Order; Product 1:N Order; Day 1:N Order. Sale, Expense y Audit amplían ese núcleo. Se conservan referencias con PROTECT y se retiran productos o anulan movimientos mediante borrado lógico.

3. Organización

config define ajustes y rutas; models define entidades; serializers transforma y valida JSON; views y auth_views exponen recursos; services concentra reglas y transacciones. static y templates forman el portal. La API versionada comienza en /api/v1/.

Autenticación y autorización

4. Consulta de seguridad con IA 1

Prompt aplicado: «Audita el registro, el login y los permisos de esta API para impedir que un cliente se convierta en dueña o acceda a pedidos ajenos».

Respuesta resumida de Codex: el filtrado de pedidos por usuario ya existía. Era necesario añadir autenticación de API por tokens, limitar credenciales inválidas y comprobar que el registro no acepte campos administrativos.

Decisión crítica: se incorporó SimpleJWT, en lugar de programar firmas propias. El registro usa create_user y solo acepta username, email y password. La dueña es un superusuario activo; ese rol nunca se asigna desde el registro público. El permiso de cada operación y el filtrado de objetos siguen siendo necesarios aunque el JWT sea válido.

Control aplicado	Evidencia verificable
Contraseña	BCrypt-SHA256 y validadores Django; mínimo 12 caracteres. Es un hash, no cifrado reversible.
JWT	Access de 5 minutos; refresh de 1 día; renovación con rotación y blacklist. Logout revoca el refresh propio.
Aislamiento	El servidor asigna el usuario del pedido. Un cliente no consulta ni cancela pedidos ajenos.
Roles	Solo dueña escribe catálogo, configura cupos, cotiza, entrega y administra finanzas.

5. Verificación y limitaciones

Las pruebas rechazan JWT vencidos o falsificados, usuarios inactivos, credenciales incorrectas y refresh de otra cuenta. Comprueban la revocación y el rechazo del refresh anterior después de renovar. También prueban la imposibilidad de elevar privilegios desde el registro.

El access ya emitido puede seguir vigente hasta cinco minutos tras logout. La sesión tradicional conserva CSRF; no se desactiva para facilitar formularios. En esta entrega la dueña tiene permisos administrativos amplios: en una operación real se recomienda un grupo limitado y MFA.

Método de trabajo con IA

Las tres consultas transcritas corresponden a revisiones estructuradas formuladas y respondidas por Codex durante la implementación solicitada por el equipo, el 30-09-2026. No se presentan como tres mensajes escritos por los alumnos ni como una auditoría externa. AUDITORIA_IA.md relaciona recomendaciones, decisiones, archivos y pruebas.

JSON, CRUD y reglas de negocio

6. Consulta de seguridad con IA 2

Prompt aplicado: «Revisa los serializadores y el CRUD: entradas inválidas, inyección SQL, manipulación de campos protegidos y acceso a objetos de otro usuario».

Respuesta resumida: mantener el ORM para evitar concatenación de SQL, validar las solicitudes y demostrar los métodos HTTP con pruebas. El titular, estado y cotización del pedido no deben ser editables por el cliente.

Decisión crítica: se usa JsonSerializer y validaciones explícitas de fecha y producto activo. Las reglas se verifican también dentro de transacciones para proteger los cupos. Se descartó borrar caracteres arbitrariamente de entradas: una búsqueda con sintaxis SQL se trata como texto. El portal muestra datos mediante textContent.

Operación	Resultado esperado
GET /api/v1/products/	200; JSON paginado, 30 por página; búsqueda y ordenamiento.
POST /api/v1/products/	201 para dueña; 401 anónimo; 403 cliente.
PUT o PATCH products/{id}/	200; validación de campos y precio.
DELETE products/{id}/	204; se retira del catálogo y conserva historial.
CRUD /api/v1/expenses/	GET, POST, PUT, PATCH y DELETE; solo dueña; anulación lógica.
Entradas y objetos	400 datos inválidos; 404 objeto no visible; 429 límite de peticiones.

7. Refactorización y consistencia

Se añadieron rutas /api/v1/ sin quitar los alias anteriores y un manejador común de errores DRF: error.status, error.code y error.details. Se eliminó el registro duplicado en views para centralizar límites y validación. Se comparten permisos y operaciones financieras para evitar divergencias.

Los pedidos cambian mediante acciones quote, confirm, cancel y deliver. Una corrección de cotización incrementa su versión y obliga a aceptarla de nuevo. Entregar genera una sola venta. Una cancelación libera cupo. Se evitó un PUT libre sobre el estado porque permitiría saltarse estas reglas.

Evidencia: pruebas de CRUD completo, JSON mal formado, precio inválido, búsqueda SQL como texto, inyección de campos protegidos, reserva sin cupo y entrega única. Son casos concretos; no certifican ausencia de todas las vulnerabilidades.

Producción y resultados

8. Consulta de seguridad con IA 3

Prompt aplicado: «Revisa la configuración de producción: secretos, CORS, HTTPS y límites de peticiones».

Respuesta resumida: separar secretos del código, restringir CORS, exigir HTTPS y agregar límites específicos de autenticación. Se aplicaron .env.example, carga de entorno, hosts explícitos y cache de throttling en base de datos.

Decisión crítica: portal y API comparten origen; no se habilita CORS universal. Se ignora X-Forwarded-For no confiable. DRF ofrece límites aproximados, no una defensa integral contra fuerza bruta o DoS. El login de sesión/admin necesita protección adicional. HSTS se limita al host, sin preload ni extensión a subdominios que el equipo no controla.

9. Evidencia de ejecución local

Comprobación	Resultado del 30-09-2026
Suite Django (SQLite temporal)	33 pruebas aprobadas en 6,861 segundos. Sin errores de sistema.
Migraciones de modelos	makemigrations --check --dry-run: sin cambios pendientes.
Producción aislada	check --deploy: sin errores; W005 y W021 por decisiones HSTS documentadas.
HTTP real con Waitress	Portada, catálogo, sesión, logo y JavaScript: 200. Finanzas anónimas: 401.
Portal	Catálogo cargado desde API y diseño revisado en navegador; sintaxis JavaScript validada.

10. Entrega y trabajo pendiente

Se incluyen código, migraciones, dependencias fijadas, .env.example sin claves, README, bitácora, pruebas y configuración Render con PostgreSQL. La carpeta .venv y las bases reales no se distribuyen. GitHub alojará el código y Render ejecutará Django con Waitress/WhiteNoise.

Estado al generar este informe: verificación local completada; publicación y prueba desde URL pública pendientes. Deben registrarse los enlaces reales y comprobarse el flujo cliente-dueña antes de presentar la entrega. No se atribuye a pruebas SQLite una validación de concurrencia en PostgreSQL.

Referencias técnicas

Django: docs.djangoproject.com/en/5.2/howto/deployment/checklist/
DRF: django-rest-framework.org/api-guide/permissions/ y /throttling/
SimpleJWT: django-rest-framework-simplejwt.readthedocs.io/en/stable/blacklist_app.html
Render: render.com/docs/free (servicio gratuito para demostración; PostgreSQL expira a los 30 días).